CMSC131: Classes, Objects, and Constructors

Chapter 6

By Naren Ayinala

May 2026

Contents

1 The Big Picture: Why Do We Need Classes? 3

2 Our First Class: Car 3

2.1 Fields, Methods, and Access Modifiers 4

3 What Is a Constructor? 4

3.1 Writing a Constructor from Scratch 5

3.2 Default Values and the Default Constructor 5

4 The `this` Keyword 6

5 Constructor Overloading and Chaining 6

5.1 Order of Constructor Execution 7

6 References, Objects, the Stack, and the Heap 7

6.1 What Happens When a Method Receives an Object? 8

7 Instance Members Versus Static Members 8

7.1 Static Is Not the Same as Public 9

8 Every Class Inherits from Object 9

8.1 `toString()`: A Readable Description 9

8.2 `equals(Object)`: Logical Equality 10

8.3 Why the Parameter Must Be `Object` 11

9 Overloading Versus Overriding 11

10 Constructors and Inheritance: A First Look at `super` 12

11 Putting Everything Together: A BankAccount Class **12**

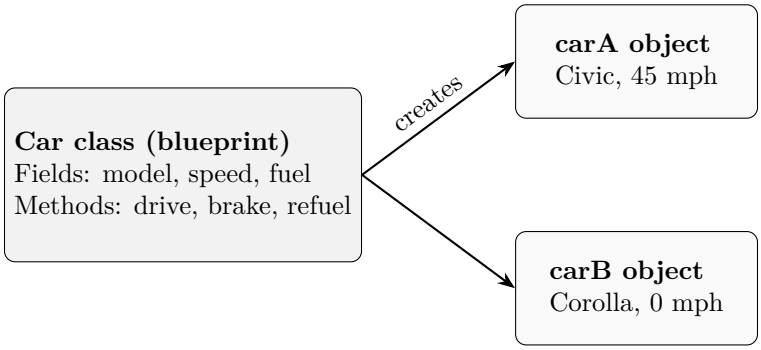
12 Tracing Objects Without Guessing **13**

1 The Big Picture: Why Do We Need Classes?

Until now, most examples have placed instructions inside `main` or other methods. Imagine a program that keeps track of 100 cars. Each car has a make, a model, a year, a speed, and a fuel level. Without classes, we might create separate variables such as `car1Speed`, `car2Speed`, and so on. We would also need to remember which variables belong together. That approach becomes difficult to maintain.

A **class** lets us describe one kind of thing. It brings together the **data** associated with that thing (fields) and the **actions** it can perform (methods). A **Car class** describes what a car object should contain and what operations it supports. A particular blue Honda Civic is an **object**, or **instance**, of that class. Another car can be created from the same class, but it has its own speed and fuel level.

Think of a class as a **blueprint** and an object as a particular thing built from that blueprint. The blueprint does not itself have a speed of 45 mph; each car object does. A class definition can be used to create many separate objects.



2 Our First Class: Car

Here is a simple class. Read it from top to bottom before worrying about every keyword.

```
public class Car {
    private String model;
    private int speed;
    private int fuel;

    public Car(String model, int fuel) {
        this.model = model;
        this.speed = 0;
        this.fuel = fuel;
    }

    public void accelerate(int amount) {
        if (amount > 0 && fuel > 0) {
            speed += amount;
            fuel--;
        }
    }

    public void brake() {
        speed = 0;
    }

    public String getModel() {
```

```
        return model;
    }

    public int getSpeed() {
        return speed;
    }
}
```

The three variables at the beginning are **instance fields**. Every Car object gets its own `model`, `speed`, and `fuel`. The methods describe what we can ask a car to do. `accelerate` updates the speed and fuel fields, while `getSpeed` returns the current speed without changing it. The block named `Car` with no return type is the **constructor**; we will examine it shortly.

We can use the class from another file called `CarDemo.java`:

```
public class CarDemo {
    public static void main(String[] args) {
        Car first = new Car("Civic", 10);
        Car second = new Car("Corolla", 20);
        first.accelerate(15);
        System.out.println(first.getSpeed()); // 15
        System.out.println(second.getSpeed()); // 0
        second.brake();
    }
}
```

The first car accelerating does not accelerate the second car. Each object has independent instance fields. Notice the dot syntax: `first.accelerate(15)` means "call the `accelerate` method on the object referenced by `first`." The variable `first` does not hold the entire object; it holds a reference to the object.

2.1 Fields, Methods, and Access Modifiers

Fields represent state: the information that may differ from one object to another. Methods represent behavior: operations that read or update that state. A `private` field is not directly accessible from unrelated classes; a `public` method can provide a controlled way to interact with it. This is part of **encapsulation**: keeping the object's internal details behind a clear interface.

```
Car car = new Car("Civic", 10);
// car.speed = -500; // Compile error: speed is private.
System.out.println(car.getSpeed()); // Legal public method.
```

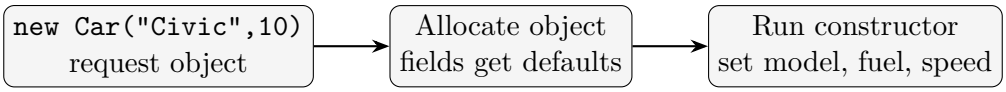
Private fields also help prevent invalid state. For instance, a method can reject negative acceleration rather than allowing arbitrary code to assign nonsense values.

3 What Is a Constructor?

A constructor is a special member used when creating an object. It has the **same name as the class** and **no return type**, not even `void`. Its purpose is to establish the object's initial state. The expression `new Car("Civic", 10)` creates a Car object and invokes a matching constructor.

The factory analogy is useful: the class is the design, `new` requests a new product, and the constructor

sets up that particular product. A constructor does not create the object all by itself: Java allocates and initializes the object as part of evaluating `new`, then runs the constructor initialization logic.



3.1 Writing a Constructor from Scratch

Suppose we want a `Book` object to remember its title and number of pages. Start with the class and fields, then write a constructor with parameters for the values we want to supply.

```
public class Book {
    private String title;
    private int pages;

    public Book(String title, int pages) {
        this.title = title;
        this.pages = pages;
    }

    public String getTitle() { return title; }
    public int getPages() { return pages; }
}
```

```
Book book = new Book("Dune", 412);
System.out.println(book.getTitle()); // Dune
System.out.println(book.getPages()); // 412
```

The parameters `title` and `pages` are local to the constructor call. The fields `this.title` and `this.pages` belong to the object and remain available after the constructor finishes. Each `new Book(...)` call initializes a separate object.

3.2 Default Values and the Default Constructor

Instance fields receive default values before constructor logic runs: numeric primitives get zero, `boolean` gets `false`, and reference fields get `null`. Local variables inside methods do *not* automatically get usable default values; they must be definitely assigned before being read.

If a class declares **no constructor at all**, Java supplies a no-argument default constructor. As soon as you declare any constructor, Java does **not** automatically supply that no-argument constructor.

```
public class Lamp {
    private boolean on;
    // No constructor written: new Lamp() is valid.
}
```

```
public class Lamp {
    private boolean on;
    public Lamp(boolean startOn) { on = startOn; }
    // new Lamp() now fails unless we add Lamp().
}
```

A constructor can also explicitly initialize fields to sensible values instead of relying on defaults. Never assume `null` is an acceptable initial value for every reference field.

4 The `this` Keyword

Inside an instance method or constructor, `this` refers to the **current object**: the object whose method or constructor is executing. It is especially useful when a parameter has the same name as a field.

```
public class Student {
    private String name;
    private int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }
}
```

On the right of `this.name = name`, the unqualified `name` means the constructor parameter. On the left, `this.name` means the field of the object being initialized. If we accidentally write `name = name`, we assign the parameter to itself, leaving the field unchanged.

```
public Student(String name) {
    name = name; // Wrong: parameter assigned to itself.
    // this.name remains null.
}
```

`this` is also useful for calling another instance method of the same object. Writing `this.brake()` is usually equivalent to writing `brake()` inside an instance method. You cannot use `this` directly in a static context such as `main`, because no current instance is associated with a static method.

5 Constructor Overloading and Chaining

Like methods, constructors can be **overloaded**: a class may have multiple constructors with different parameter lists. This lets callers create an object with different amounts of information.

```
public class Car {
    private String model;
    private int speed;
    private int fuel;

    public Car(String model, int fuel) {
        this.model = model;
        this.speed = 0;
        this.fuel = fuel;
    }

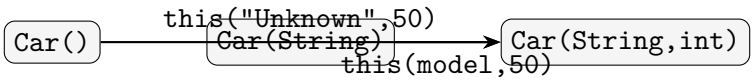
    public Car(String model) {
        this(model, 50);
    }

    public Car() {
        this("Unknown", 50);
```

```
    }
}
```

The call `this(model, 50)` does **not** call an ordinary method. It invokes another constructor of the *same object*. This is **constructor chaining**. All three ways of creating a Car eventually run the two-argument constructor, so the initialization logic stays in one place.

```
Car a = new Car("Civic", 20); // two-argument constructor
Car b = new Car("Civic"); // one-argument, then two-argument
Car c = new Car(); // no-argument, then two-argument
```



In traditional Java constructor syntax, an explicit `this(...)` constructor invocation must be the first statement. A constructor cannot invoke itself, directly or indirectly, in a cycle. A `this(...)` constructor call and a `super(...)` constructor call cannot both appear as explicit constructor invocations in the same constructor.

5.1 Order of Constructor Execution

Consider the following class. Each print statement tells us which constructor is running.

```
public class Ticket {
    public Ticket() {
        this(5);
        System.out.println("No-arg finished");
    }
    public Ticket(int number) {
        System.out.println("Number: " + number);
    }
}
// new Ticket() prints:
// Number: 5
// No-arg finished
```

The no-argument constructor delegates to the integer constructor. The integer constructor finishes first; only then does execution resume after `this(5)`. This is why chaining is best understood as a series of calls, not as three constructors executing independently.

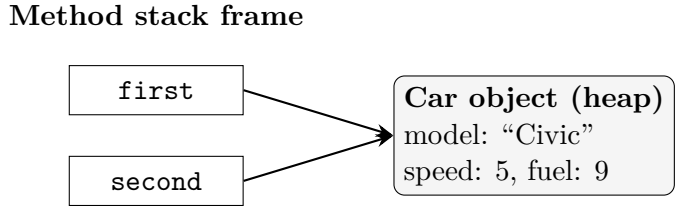
6 References, Objects, the Stack, and the Heap

For the beginner mental model used in CMSC131, **local variables and parameters** belong to a method's stack frame, while objects created with `new` live on the **heap**. A reference variable contains a reference to an object, not the object's fields themselves. Exact JVM optimizations and physical layouts can differ, but this model correctly explains the behavior of our Java examples.

```
Car first = new Car("Civic", 10);
Car second = first;
```

```
second.accelerate(5);
System.out.println(first.getSpeed()); // 5
```

There is only **one** Car object in this example. Assignment `second = first` copies the reference; it does not create a second Car. Both variables point to the same object, so changes through `second` are visible through `first`.



To make an independent second car, construct one with `new`. Do not confuse `Car b = a;` with `Car b = new Car(...);`. The former copies a reference; the latter creates another object.

6.1 What Happens When a Method Receives an Object?

Java is **always pass-by-value**. When an object reference is passed to a method, the *reference value* is copied into the parameter. The method can mutate the shared object through that copied reference, but reassigning its parameter does not reassign the caller's variable.

```
public static void replaceCar(Car c) {
    c = new Car("Truck", 30);
    c.accelerate(10);
}
public static void speedUp(Car c) {
    c.accelerate(10);
}
// In main:
Car myCar = new Car("Civic", 10);
replaceCar(myCar);
System.out.println(myCar.getSpeed()); // 0
speedUp(myCar);
System.out.println(myCar.getSpeed()); // 10
```

`replaceCar` changes only its local parameter to point to another object. `speedUp` calls a method on the shared object, so the caller sees the change.

7 Instance Members Versus Static Members

An **instance field** belongs to a particular object; a **static field** belongs to the class as a whole. If we want every Car to have its own speed but want to count how many Cars have been created, speed should be an instance field and the count can be static.

```
public class Car {
    private int speed;
    private static int carsCreated = 0;

    public Car() {
```

```
        speed = 0;
        carsCreated++;
    }
    public int getSpeed() { return speed; }
    public static int getCarsCreated() { return carsCreated; }
}
```

```
Car a = new Car();
Car b = new Car();
System.out.println(Car.getCarsCreated()); // 2
```

The preferred syntax for calling a static method is `ClassName.methodName()`. An instance method normally requires an object reference, such as `a.getSpeed()`. A static method cannot directly access `speed` or `this`, because there is no particular Car object selected as the current instance.

Feature	Instance member	Static member
Belongs to	One object	Class
Typical call	`car.getSpeed()`	`Car.getCarsCreated()`
Can use `this`?	Yes, in instance context	No
Copies	One instance field per object	One class-level field

7.1 Static Is Not the Same as Public

`public` controls accessibility, while `static` controls whether a member is associated with the class or with an instance. These are separate ideas. A field can be `private static`, and a method can be `public` without being static. Do not read `public static` as one inseparable keyword.

8 Every Class Inherits from Object

Every ordinary Java class ultimately inherits from `java.lang.Object`. The `Object` class supplies methods such as `toString()`, `equals(Object)`, and `hashCode()`. We can **override** these methods to provide behavior that makes sense for our class.

8.1 `toString()`: A Readable Description

If we print an object without overriding `toString()`, the default text usually looks like a class name followed by a hexadecimal hash representation, not the object's fields. Overriding `toString()` lets us choose a meaningful description.

```
public class Dog {
    private String name;
    private int age;

    public Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
```

```
    public String toString() {
        return name + " (age " + age + ")";
    }
}
```

```
Dog d = new Dog("Milo", 3);
System.out.println(d); // Milo (age 3)
System.out.println(d.toString()); // Milo (age 3)
```

`System.out.println(d)` uses a String representation of `d`, which invokes its `toString()` implementation. The `@Override` annotation asks the compiler to check that we are truly overriding a method inherited from a superclass or interface.

8.2 `equals(Object)`: Logical Equality

For object references, `==` asks whether two variables refer to the **same object**. By default, `Object.equals()` also behaves like identity equality. We can override `equals` to define **logical equality**: two different Dog objects may count as equal if their names and ages match.

```
import java.util.Objects;
public class Dog {
    private String name;
    private int age;
    public Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof Dog)) return false;
        Dog dog = (Dog) other;
        return age == dog.age && Objects.equals(name, dog.name);
    }
    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }
}
```

This example first checks whether both references already point to the same object. Next it rejects values that are not Dogs (including `null`). Then it compares the primitive `age` values with `==` and the String names by content using `Objects.equals`, which also handles `null` safely.

```
Dog a = new Dog("Milo", 3);
Dog b = new Dog("Milo", 3);
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

Important: when overriding `equals()`, also override `hashCode()` so equal objects have equal hash codes. This matters when objects are stored in hash-based collections. The example uses `Objects.hash` to keep the two definitions consistent.

8.3 Why the Parameter Must Be `Object`

The inherited method has signature `boolean equals(Object other)`. Writing `equals(Dog other)` creates a different overloaded method; it does not override `Object.equals`. The `@Override` annotation helps catch this mistake at compile time.

```
// Not an override of Object.equals:
public boolean equals(Dog other) {
    return other != null && age == other.age;
}
```

That overloaded method may still be callable, but a general API expecting `equals(Object)` will not use it as a replacement for the inherited method.

9 Overloading Versus Overriding

These two terms sound similar but describe different ideas. **Overloading** means multiple methods (or constructors) share a name but have different parameter lists. The compiler chooses which overload matches the arguments. **Overriding** means a subclass provides its own implementation of an inherited instance method with a compatible signature.

```
public class Printer {
    public void print(int n) { System.out.println(n); }
    public void print(String s) { System.out.println(s); }
    // Two overloads: different parameter types.
}
```

```
public class Animal {
    public String sound() { return "?"; }
}
public class Cat extends Animal {
    @Override
    public String sound() { return "meow"; }
}
```

```
Animal pet = new Cat();
System.out.println(pet.sound()); // meow
```

The last example demonstrates **dynamic dispatch**: the runtime object is a Cat, so Cat's overridden instance method executes even though the variable type is Animal. Static methods are *hidden*, not overridden in this polymorphic sense. Constructors are not inherited and cannot be overridden.

Question	Overloading	Overriding
Where?	Often same class	Subclass and superclass
Parameters	Must differ	Compatible inherited signature
Purpose	Different inputs	Change inherited behavior
Chosen when?	Primarily compile time	Runtime dispatch for instance methods
Constructors?	Can be overloaded	Cannot be overridden

10 Constructors and Inheritance: A First Look at `super`

When one class extends another, the subclass constructor must initialize the superclass part of the object. The syntax `super(...)` invokes a superclass constructor. Unlike `this(...)`, which delegates to another constructor in the same class, `super(...)` delegates to a constructor in the parent class.

```
public class Vehicle {
    protected String brand;
    public Vehicle(String brand) {
        this.brand = brand;
        System.out.println("Vehicle constructor");
    }
}
public class Bus extends Vehicle {
    private int seats;
    public Bus(String brand, int seats) {
        super(brand);
        this.seats = seats;
        System.out.println("Bus constructor");
    }
}
// new Bus("Volvo", 40) prints:
// Vehicle constructor
// Bus constructor
```

The superclass constructor runs before the remainder of the subclass constructor body. If you do not write `super(...)` or `this(...)` in a traditional constructor, Java implicitly tries to call the superclass's no-argument constructor. If the parent class has no accessible no-argument constructor, you must explicitly select a valid superclass constructor.

11 Putting Everything Together: A BankAccount Class

This example uses fields, validation, overloaded constructors, `this`, a static counter, and `toString()` together.

```
public class BankAccount {
    private String owner;
    private double balance;
    private static int accountsCreated = 0;

    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = Math.max(0, balance);
        accountsCreated++;
    }
    public BankAccount(String owner) {
        this(owner, 0);
    }
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }
```

```
    public double getBalance() { return balance; }
    public static int getAccountsCreated() {
        return accountsCreated;
    }
    @Override
    public String toString() {
        return owner + ": $" + balance;
    }
}
```

```
BankAccount a = new BankAccount("Ada", 100);
BankAccount b = new BankAccount("Grace");
a.deposit(25);
System.out.println(a.withdraw(50)); // true
System.out.println(a.getBalance()); // 75.0
System.out.println(b.getBalance()); // 0.0
System.out.println(BankAccount.getAccountsCreated()); // 2
System.out.println(a); // Ada: $75.0
```

The balance is private so outside code cannot directly set it to a negative number. The one-argument constructor delegates to the two-argument constructor. The static counter belongs to the class rather than to either account. Finally, `toString()` controls the text shown when the account is printed. For real financial applications, monetary amounts should not generally be modeled using binary floating-point `double`; this example uses it only to keep the introductory class mechanics clear.

12 Tracing Objects Without Guessing

When working through a class problem, first identify which statements create new objects. Draw one heap box per `new` expression. Then draw arrows for reference variables. Next mark which fields are instance fields and which are static. Finally trace method calls and field updates in order. Do not draw a new object merely because one reference variable is assigned to another.

```
Car a = new Car("Civic", 10);
Car b = a;
Car c = new Car("Civic", 10);
a.accelerate(5);
System.out.println(a == b); // true
System.out.println(a == c); // false
System.out.println(b.getSpeed()); // 5
System.out.println(c.getSpeed()); // 0
```

There are exactly two Car objects: one shared by `a` and `b`, and one referenced by `c`. Identical constructor arguments do not cause Java to reuse a Car object. Only explicit sharing of references makes two variables point to the same instance.